

Blue Sheild		보안 오케스트레이션	
개요			
구성	▪ 보안 오케스트레이션 구성		
	구분	Enterprise 대규모 프로젝트(향후)	
	1. 로그 수집(Ingestion)	Filebeat	
	2. 데이터 저장/분석(SIEM)	SQLite 또는 텍스트 파일, Elastic Stack (ELK)	
	3. 탐지 엔진 (Detection)	Suricata + 머신러닝 이상 탐지(UEBA)	
	4. 자동화 (SOAR)	Shuffle	
	5. 외부 연동(TI-Threat Intelligence)	VirusTotal, MISP (위협 인텔리전스)	
6. 관제 화면(Dashboard)	Kibana,		
로그 수집, 데이터 저장/분석	cyber-security-Datasets https://www.kaggle.com/discussions/general/335189		
탐지 엔진	eve.json 생성 ↓ Python 또는 SQLite로 이벤트 읽기 ↓ 탐지 룰 적용 ↓ 위험 이벤트 판정 ↓ 자동 대응 스크립트 실행 ↓ 결과를 Streamlit에서 시각화		
	suricata + 2차 GRU로 탐지 하여 걸러냄(학습된 모델)		
	▪ 탐지 대상 이벤트 설정		
	1) 포트 스캔 탐지		
	2) 짧은 시간 내 다수 IP 접속 탐지		
	3) 특정 위험 시그니처 alert 탐지		
	4) 비정상적으로 많은 DNS 요청 탐지		
	5) 같은 IP의 반복 실패 행위 탐지		

- Suricata의 로그 eve.json Sample

```
{ "timestamp": "2026-03-13T10:10:10.123456+0900",
  "event_type": "alert",
  "src_ip": "192.168.0.10",
  "dest_ip": "8.8.8.8",
  "proto": "TCP",
  "alert": {
    "severity": 2,
    "signature": "ET SCAN Nmap Scripting Engine User-Agent Detected"
  }
}
```

- 탐지 스크립트 detect_engine.py

```
import json
from collections import defaultdict
from datetime import datetime
EVE_FILE = "eve.json"
OUTPUT_FILE = "alerts.json"
alerts_found = []
ip_counter = defaultdict(int)
def parse_time(ts):
    try:
        return datetime.fromisoformat(ts.replace("+0900", "+09:00"))
    except Exception:
        return None
with open(EVE_FILE, "r", encoding="utf-8") as f:
    for line in f:
        line = line.strip()
        if not line:
            continue
        try:
            event = json.loads(line)
        except json.JSONDecodeError:
            continue
        event_type = event.get("event_type")
        src_ip = event.get("src_ip", "unknown")
        dest_ip = event.get("dest_ip", "unknown")
        ts = event.get("timestamp")
        # 규칙 1: Suricata alert 이벤트
        if event_type == "alert":
            alert_info = event.get("alert", {})
            severity = alert_info.get("severity", 99)
            signature = alert_info.get("signature", "unknown")
            if severity <= 3:
                alerts_found.append({
```

```

        "rule": "high_priority_alert",
        "timestamp": ts,
        "src_ip": src_ip,
        "dest_ip": dest_ip,
        "severity": severity,
        "signature": signature
    })

    # 규칙 2: source IP별 접속 횟수 카운트
    if src_ip != "unknown":
        ip_counter[src_ip] += 1
# 규칙 3: 지나치게 많은 요청을 보낸 IP
for ip, count in ip_counter.items():
    if count >= 50:
        alerts_found.append({
            "rule": "too_many_requests",
            "src_ip": ip,
            "count": count
        })
with open(OUTPUT_FILE, "w", encoding="utf-8") as f:
    json.dump(alerts_found, f, indent=2, ensure_ascii=False)
print(f"[+] 탐지 완료: {len(alerts_found)}개 이벤트 저장")

```

▪ 결과 SQLite에 저장

```

CREATE TABLE detections (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    timestamp TEXT,
    rule_name TEXT,
    src_ip TEXT,
    dest_ip TEXT,
    severity INTEGER,
    signature TEXT,
    extra_data TEXT
);

```

- FastAPI 연동 및 Docker Image 빌드
 - 완성된 .pt 모델 파일을 FastAPI로 감싸 엔드포인트(/analyze) 생성.
 - Dockerfile을 작성하여 ai_engine 컨테이너로 도커 망에 합류.

자동화(SOA
R)

Shuffle 설치 및 플레이북(Playbook) 작성

- Shuffle 컨테이너 세팅 및 Webhook 트리거 생성.
- 탐지 결과가 나오면 자동으로 대응 스크립트를 실행
- soar_runner.py
 - 1) alerts.json 읽기
 - 2) 위험도 판단

- 3) 대응 액션 선택
- 4) 로그 기록
- 5) 필요에 따라 알림 출력

▪ soar_runner.py Sample

```
import json
from datetime import datetime
ALERT_FILE = "alerts.json"
BLACKLIST_FILE = "blacklist.txt"
INCIDENT_LOG = "incident.log"
def write_incident(message):
    with open(INCIDENT_LOG, "a", encoding="utf-8") as f:
        f.write(f"[{datetime.now().isoformat()}] {message}\n")
def add_to_blacklist(ip):
    with open(BLACKLIST_FILE, "a", encoding="utf-8") as f:
        f.write(ip + "\n")
with open(ALERT_FILE, "r", encoding="utf-8") as f:
    alerts = json.load(f)
for alert in alerts:
    rule = alert.get("rule")
    src_ip = alert.get("src_ip", "unknown")
    if rule == "high_priority_alert":
        write_incident(f"HIGH ALERT from {src_ip}: {alert}")
        add_to_blacklist(src_ip)
    elif rule == "too_many_requests":
        write_incident(f"SCAN SUSPECT from {src_ip}: {alert}")
        add_to_blacklist(src_ip)
print("[+] 자동 대응 완료")
```

- 룰별 대응처리를 위한 PlayBook.py 적용
 - 1) 어떤 탐지 규칙이 발생하면 작동하는가
 - 2) 어떤 액션들을 순서대로 수행시키는가

▪ 향후 Enterprise 구조로 확장에 응용 가능

```
PLAYBOOK = {
    "high_priority_alert": ["log_incident", "blacklist_ip"],
    "too_many_requests": ["log_incident", "blacklist_ip"]
}
```

외부 연동(TI)

- 현재 Mini 단계에서 로컬 위협 인텔리전스(TI) 기반으로 구현
- 처음부터 VirusTotal API를 사용하는 것이 아닌 임의의 json Sample 활용

```
{ "malicious_ips": ["10.10.10.10", "123.45.67.89"],  
  "malicious_domains": ["bad-example.com"]  
}
```

- 탐지 결과의 src_ip가 목록에 있다면?
 - 1) 위험도 높임
 - 2) 대시보드에 TI 매칭 표시
- 확장을 진행할 때 VirusTotal, AbuseIPDB, MISP와 같은 외부 서비스 연동 가능하도록 설계

탐지 발생
↓
src_ip 추출
↓
외부 평판 조회
↓
악성 판정이면 우선순위 상승
↓
자동 대응 강화

- 로그, 탐지 결과, 대응 결과를 한 화면에 표기
- Mini 단계에선 Streamlit 활용 및 Sample 화면
 - 1) 총 이벤트 수
 - 2) alert 이벤트 수
 - 3) 탐지된 이상행위 수
 - 4) 위험 IP 목록
 - 5) 최근 incident 로그
 - 6) 탐지 결과 테이블

관제 화면

[총 이벤트] 1200
[탐지 이벤트] 24
[차단 후보 IP] 3

최근 탐지 목록

- 192.168.0.5 / high_priority_alert
- 10.0.0.8 / too_many_requests

최근 대응 로그

- blacklist 추가
- incident 기록

```

import streamlit as st
import json
import os
st.title("Mini SOC Dashboard")
alerts = []
if os.path.exists("alerts.json"):
    with open("alerts.json", "r", encoding="utf-8") as f:
        alerts = json.load(f)
blacklist = []
if os.path.exists("blacklist.txt"):
    with open("blacklist.txt", "r", encoding="utf-8") as f:
        blacklist = [line.strip() for line in f if line.strip()]
st.metric("탐지 이벤트 수", len(alerts))
st.metric("차단 대상 IP 수", len(set(blacklist)))
st.subheader("탐지 결과")
st.json(alerts)
st.subheader("차단 대상 IP")
st.write(sorted(set(blacklist)))

```

최종 구현 예상 폴더

```

mini_soc/
├── eve.json
├── detect_engine.py
├── soar_runner.py
├── dashboard.py
├── alerts.json
├── blacklist.txt
├── incident.log
├── rules.json
└── ti_feed.json

```

- detect_engine.py : 로그를 읽고 이상행위 탐지
- soar_runner.py : 탐지 결과 보고 자동 대응
- dashboard.py : 결과 시각화
- rules.json : 탐지 임계값 저장
- ti_feed.json : 악성 IP/도메인 목록